



BETABYTEZ

crafting digital magic

SUMMER INTERNSHIP PROGRAM 2026

Official Task Book

AI / ML Engineering

Program	BetaBytez Summer Internship 2026
Duration	6 Weeks
Tracks	AI / ML (Full Stack)
Tasks	5 Progressive Tasks Per Track
Submission	Weekly GitHub Repository Links
Final Week	Week 6 Capstone Project & Demo
Mode	Remote
Issued By	Technical Team, BetaBytez
Year	2026

This document is confidential and issued exclusively to selected BetaBytez interns.

www.betabytez.com · infobetabytez@gmail.com

This track takes you from training your first simple machine learning model all the way to building a deployed, full-stack AI platform. Each task introduces new concepts, tools, and complexity. By Task 5, you will have built a complete AI-powered application you can confidently present in interviews and graduate school applications.

Tech Stack: Python, Scikit-learn, Pandas, TensorFlow/PyTorch, FastAPI/Flask/Streamlit, LangChain, Vector Databases

TASK 1 | AI / ML

Train Your First ML Model with a Simple UI

Week	Week 1
Difficulty	Beginner
Tech	Python, Pandas, Scikit-learn, Streamlit or Flask

 Problem Statement

Before you can build intelligent systems, you must understand the fundamentals of how a machine learning model is trained, evaluated, and used to make predictions. In this task, you will train a classification model on a real dataset and wrap it in a simple, usable web interface — so that a non-technical person can interact with your model without touching any code.

 Detailed Description

Complete the following steps:

1. Choose a publicly available classification dataset from Kaggle or UCI ML Repository (examples: Iris, Titanic survival, Heart Disease, Spam SMS). Do not use MNIST.
2. Perform Exploratory Data Analysis (EDA): inspect the data, check for missing values, understand the distribution of features and the target label.
3. Preprocess the data: handle missing values, encode categorical variables, scale numerical features where needed.
4. Train at least two different classification models (e.g., Logistic Regression and Random Forest) and compare their performance.
5. Evaluate both models using: Accuracy, Precision, Recall, and F1-Score. Write a 3–5 sentence explanation of which model performed better and why.
6. Save the best model using joblib or pickle.
7. Build a simple web UI using Streamlit (preferred) or Flask where a user can input feature values and get a prediction from your saved model.

All steps must be in a well-commented Jupyter Notebook. The notebook is your working document — the Streamlit/Flask app is the deliverable that non-technical users would see.

Deliverables

1	Jupyter Notebook with complete EDA, preprocessing, model training, and evaluation — clean and commented.
2	Saved model file (.pkl or .joblib) included in the repo.
3	Working Streamlit or Flask app that loads the saved model and takes user input to return a prediction.
4	GitHub repo named: betabytez-aiml-task1-yourname with README explaining the dataset, approach, and how to run the app.

T A S K 2 | AI / ML

Model Training on Two Datasets with Comparison

Week	Week 2
Difficulty	Beginner–Intermediate
Tech	Python, Scikit-learn, Pandas, Matplotlib, Seaborn

Problem Statement

In the real world, data scientists rarely work with a single clean dataset. They must compare approaches, understand why a model performs differently on different data, and communicate their findings clearly to non-technical stakeholders. In this task, you will train models on two different datasets and produce a structured comparative analysis report.

Detailed Description

Select two datasets from different domains (e.g., one medical, one financial — or one text-based, one numerical). Both must be classification problems. For each dataset:

8. Perform full preprocessing and EDA with visualizations (at least 3 plots per dataset: distribution, correlation heatmap, class balance chart).
9. Train a minimum of two models on each dataset.
10. Evaluate using Accuracy, Precision, Recall, F1-Score, and a Confusion Matrix visualization.
11. Apply at least one technique to improve the weaker model: hyperparameter tuning (GridSearchCV), feature selection, or handling class imbalance (SMOTE or class_weight).

Then produce a Comparative Analysis Report (inside the notebook, in markdown cells) that answers: Which dataset was harder to model and why? Which model generalized better across both datasets? What did you learn about the relationship between data quality and model performance?

Deliverables

1	One Jupyter Notebook containing both dataset analyses, all models, visualizations, and the written comparative report.
2	At least 6 visualizations total (3 per dataset) — properly labeled and titled.
3	GitHub repo named: betabytez-aiml-task2-yourname.
4	README summarizing the two datasets chosen and the key finding from the comparison.

T A S K 3 | AI / ML

End-to-End Prediction System with Web Interface

Week	Week 3–4
Difficulty	Intermediate
Tech	Python, Scikit-learn or TensorFlow, FastAPI or Flask, React or Streamlit

Problem Statement

A model that lives only in a Jupyter Notebook is not useful in the real world. Data scientists are increasingly expected to deploy their models as usable services. In this task, you will build a complete, end-to-end prediction system — from a trained model all the way to a working web application where any user can get predictions in real time.

Detailed Description

Choose a real-world prediction problem from one of these domains:

- House/property price prediction (regression)
- Student performance or exam score prediction (regression)
- Customer churn prediction (classification)
- Loan default risk prediction (classification)

Build the following pipeline:

12. Data collection, preprocessing, and feature engineering — document every transformation decision.
13. Train the best model you can for this problem — experiment with at least three approaches.
14. Build a FastAPI or Flask backend that exposes a /predict endpoint — the endpoint accepts JSON input and returns a prediction with a confidence score.
15. Build a frontend (Streamlit or a basic React form) that sends user inputs to the API and displays the prediction result clearly.
16. Add basic input validation — if a user sends invalid data, the API must return a meaningful error message, not crash.

The system must work end-to-end: a user fills in the form, the frontend calls the API, the API runs the model, and the prediction appears on screen. Document every step in the README.

Deliverables

1	Working prediction API with a /predict endpoint, tested and returning valid predictions.
2	Frontend form connected to the API with clear prediction output display.
3	GitHub repo named: betabytez-aiml-task3-yourname with /model, /api, and /frontend folders.
4	README with API documentation (endpoint, input format, example request/response) and frontend setup instructions.

T A S K 4 | AI / ML

RAG-Based Document Q&A Chatbot

Week	Week 4–5
Difficulty	Advanced
Tech	Python, LangChain or LlamaIndex, FAISS or ChromaDB, LLM API, Streamlit

Problem Statement

General-purpose AI chatbots often hallucinate — they make up answers from their training data rather than from actual facts. Retrieval-Augmented Generation (RAG) solves this by forcing the AI to answer only from specific, real documents you provide. RAG-based systems are one of the most in-demand skills in the AI industry right now. In this task, you will build a working RAG chatbot grounded in a document set of your choice.

Detailed Description

Choose a document set to power your chatbot (examples: university course slides, a technical manual, a set of research papers, BetaBytez program documentation, or any domain you find interesting). Then build the following pipeline:

17. Document Ingestion: load your documents, split them into chunks (consider chunk size and overlap carefully — explain your choice in the README).
18. Embedding & Indexing: generate embeddings for each chunk and store them in a vector database (FAISS or ChromaDB — both have free local options).
19. Retrieval: given a user query, retrieve the top-k most relevant chunks using vector similarity search.
20. Generation: pass the retrieved chunks + user query to an LLM (use a free-tier API or an open model) and generate a grounded answer.
21. Grounding check: if the answer is not in the documents, the chatbot must say so — it must not hallucinate an answer from general knowledge.
22. Build a chat-style UI in Streamlit where users can ask questions and see both the answer and the source chunk(s) it came from.

Test your system with at least 5 questions — 3 that the documents can answer and 2 that they cannot. Document the results honestly in your README.

Deliverables

1	Working RAG chatbot that answers questions from your document set with source citations shown.
2	Honest test results: 3 in-scope and 2 out-of-scope questions, with the chatbot's actual responses documented.
3	GitHub repo named: betabytez-aiml-task4-yourname.
4	README explaining: document set chosen, chunking strategy, embedding model used, vector DB choice, and LLM used — with reasoning for each decision.

T A S K 5 | AI / ML

CAPSTONE — Full-Fledged AI Platform

Week	Week 6 — Final Submission
Difficulty	Advanced — Capstone
Tech	Python, ML/DL, FastAPI, React or Streamlit, Vector DB, LLM API, Deployment

Problem Statement

Your capstone is a complete, deployed AI platform that combines multiple skills from Tasks 1–4 into one cohesive, real-world system. This is not a notebook project — it is a full application with a backend, a frontend, at least one trained or fine-tuned model, and AI-powered features that deliver genuine value to a real user.

Detailed Description

Choose ONE of the following platforms to build:

- Option A: AI-Powered Resume Screener — upload job description + multiple resumes, system ranks candidates by fit, extracts matched skills, and generates a short AI summary for each candidate.
- Option B: Intelligent Study Assistant — upload course documents, the assistant generates summaries, quiz questions, and answers user questions via RAG, with a progress-tracking dashboard.
- Option C: Smart Data Analysis Platform — upload any CSV dataset, the platform performs automated EDA, generates charts, and allows users to ask natural language questions about the data.

Regardless of which option you choose, your platform must include:

23. At least one trained or fine-tuned ML model OR a RAG pipeline as the core AI engine.
24. A FastAPI or Flask backend exposing the AI features as API endpoints.
25. A frontend (React or Streamlit) with a clean, usable interface — not just raw outputs.
26. At least one data visualization in the application (chart, graph, or analytics view).
27. Deployed on free-tier services (Render/Railway for backend, Vercel/Netlify/Streamlit Cloud for frontend).
28. A complete system architecture diagram included in the README.

Deliverables

1	Live deployed AI platform accessible via a public URL — frontend and backend both live.
2	GitHub repo named: betabytez-aiml-task5-yourname with clean structure and full commit history.
3	README including: platform overview, architecture diagram, features list, setup instructions, and live URL.
4	5–7 minute Loom or screen-recorded demo video — mandatory for certification. Walk through every feature and explain your technical decisions.
5	This is your flagship AI portfolio project. It should be something you are proud to show in an interview.